

Reviewer Pack — Minimal Rank of Symmetric Functions Bounds XOR Circuit Weigh...

Entry d2989d184858 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 15:43:53 UTC

Minimal Rank of Symmetric Functions Bounds XOR Circuit Weights

Entry ID: d2989d184858 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 15:43:53 UTC

1. Conjecture

Field A (mathematical branch): Symmetric Function Theory **Field B** (complexity object): Complexity Theory: Boolean Function Complexity

Statement:

[‘For every XOR circuit C with n inputs and size S , there exists a symmetric function f over n variables such that the rank of its Hessian matrix is at least $\Omega(S^{1/4})$.’, ‘Equivalently, for any given n , there is an absolute constant $c > 0$ such that for all XOR circuits C with size S , the maximum weight of any Boolean function computed by C is at most $O(S^c)$.’, ‘If a counterexample exists with a symmetric function f and XOR circuit C of size $n \leq 40$ where the rank of the Hessian matrix is less than $c \cdot (n/S)^{1/4}$, it falsifies the conjecture.’]

Rationale (proposer’s reasoning):

[‘Symmetric functions have been extensively studied in algebraic combinatorics, but their connection to complexity theory, particularly circuit complexity, remains underexplored.’, ‘The use of symmetric functions in bounding circuit weights could potentially reveal new insights into the structure of Boolean functions and lead to improved lower bounds on circuit complexity.’, ‘Given the known lower bounds for monotone circuits, extending this concept to XOR circuits via symmetric functions might yield a different perspective on the computational power of such circuits.’]

Taxonomy category: MONOTONE_CLIQUE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c9bdb88d979c2416

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a symmetric function f and XOR circuit C of size $n \leq 40$, the conjecture is supported if the rank of the Hessian matrix of f is at least $c(n/S)^{1/4}$, where S is the size of C and c is an absolute constant greater than 0. The conjecture is falsified if there exists a counterexample with the rank being less than $c(n/S)^{1/4}$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_xor_circuit(n):
        circuit = []
        for _ in range(random.randint(5, 20)):
            gate = ('XOR', [random.choice(['x' + str(i) for i in range(n)]) for _ in range(2)])
            circuit.append(gate)
        return circuit

    def construct_symmetric_function(circuit):
        n = len([var for var in circuit[0][1] if 'x' in var])
        coeffs = [random.uniform(-1, 1) for _ in range(2**n)]
        return coeffs

    def hessian_matrix(f, vars):
        n = len(vars)
        H = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(i, n):
                delta_i = {vars[i]: 1} if i == j else {vars[i]: -1}
                delta_j = {vars[j]: 1} if i == j else {vars[j]: -1}
                H[i][j] = (f(delta_i) - f(**delta_i, **delta_j)) / 4
                if i != j:
                    H[j][i] = H[i][j]
        return H

    def rank(matrix):
        m, n = len(matrix), len(matrix[0])
        A = [row[:] for row in matrix]
```

```

r = 0
for j in range(n):
    i_max = max(range(r, m), key=lambda i: abs(A[i][j]))
    if abs(A[i_max][j]) < 1e-9:
        continue
    A[r], A[i_max] = A[i_max], A[r]
    for i in range(r + 1, m):
        factor = -A[i][j] / A[r][j]
        for k in range(n):
            A[i][k] += factor * A[r][k]
    r += 1
return r

def max_weight(circuit):
    n = len([var for var in circuit[0][1] if 'x' in var])
    weights = [random.uniform(0, 1) for _ in range(2**n)]
    return max(weights)

n = random.randint(5, 40)
circuit = generate_xor_circuit(n)
f_coeffs = construct_symmetric_function(circuit)
H = hessian_matrix(lambda x: sum(f_coeffs[i] * prod(x[var] for var in vars) for i, vars in enumerate(f_coeffs)))
rank_H = rank(H)
max_weight_C = max_weight(circuit)

c = 1.0
conjecture_holds = rank_H >= c * (n / len(circuit))**(1/4)
counterexample = "" if conjecture_holds else f"rank_H={rank_H}, expected >= {c * (n / len(circuit))**(1/4)}"

return {
    "metric_name": "Rank of Hessian Matrix",
    "metric_value": rank_H,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_rank = sum(r["metric_value"] for r in results) / len(results)
    std_rank = math.sqrt(sum((r["metric_value"] - mean_rank)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_rank} std={std_rank} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_rank} std={std_rank} support_fraction={support_fraction}")
    else:

```

```
first_failing_seed = next((seed for seed, r in zip(seeds, results) if not r["conjecture_holds"]), None)
print(f"RESULT: FALSIFIED counterexample={r['counterexample']} first_failing_seed={first_failing_seed}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

- -STDERR- -

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4f32d5d5.py", line 91, in <module>

```
result = run_trial(seed)
```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4f32d5d5.py", line 69, in run_trial

```
H = hessian_matrix(lambda x: sum(f_coeffs[i] * prod(x[var] for var in vars) for i, vars in enumerate
```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4f32d5d5.py", line 40, in hessian_matrix

```
H[i][j] = (f(delta_i) - f(**delta_i, **delta_j)) / 4
```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4f32d5d5.py", line 69, in <lambda>

```
H = hessian_matrix(lambda x: sum(f_coeffs[i] * prod(x[var] for var in vars) for i, vars in enumerate
```

File "/home/ludo/Scrivania/SEC/research/pvsnp sandbox/test 4f32d5d5.py", line 69, in <genexpr>

```
H = hessian matrix(lambda x: sum(f coeffs[i] * prod(x[var] for var in vars) for i, vars in enumerate
```

```
NameError: name 'prod' is not defined
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from verifying the conjecture's conditions. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12004
2	preregistration	ollama_remote	glm4:latest	0	0	5940
3	novelty	ollama_remote	glm4:latest	0	0	4911
4	novelty	ollama_remote	glm4:latest	0	0	5055
5	test gen	ollama_remote	qwen2.5-coder:7b	0	0	16878

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12556
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11962
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11959
9	judge	ollama_remote	glm4:latest	0	0	10627

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 91893 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/d2989d184858.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/d2989d184858.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/d2989d184858.tar.gz (if generated)